Project for the Subject: Large Dataset Management

Project Title:

Design of a Cloud-Based System for MongoDB Performance Analysis Using JMeter.
Proving the Hypothesis on the Significance of Performance Metric Differences.

Table of Contents

Netflix TV Shows and Movies — Technical and Integration Layer

- Configuration of connection to cloud-based MongoDB Atlas database
- Preparation of database structure and document collections (titles, credits)
- Importing data from CSV files into MongoDB
- Implementation of test queries to the database (find() operations)
- Integration of the Python environment with the database using the PyMongo library

Performance Analysis and Results Processing

- Preparation of performance test scenarios for different data collections
- Measurement of database query response times
- Calculation of statistical measures (mean, median)
- Conducting statistical tests (Student's t-test, Welch's test) to verify the research hypothesis
- Visualization of analysis results (response time charts)
- Preparation of the reporting section covering results and conclusions
- Preliminary verification of data correctness and queries

1. Project Objective

The objective of the project was to design and implement a cloud-based MongoDB database performance analysis system, enabling:

- measurement of response times for various database operations,
- comparison of query performance executed on different document collections,
- statistical verification of the hypothesis on the significance of performance metric differences.

The project was experimental-analytical in nature and was implemented exclusively using public and free cloud services.

2. Description of Information System Requirements

Functional Scope

- Connection to cloud MongoDB database
- Import and storage of document data
- Provision of an API for executing test queries
- Measurement of response times
- Statistical analysis of results (mean, median, Student's t-test, Welch's test)

Limitations

- Use of free-tier services only
- Execution environment: Google Colab
- Performance limitations imposed by the free-tier MongoDB Atlas cluster

3. Technology Stack Description

Database Management System

MongoDB Atlas was used — a cloud version of the non-relational MongoDB database, characterized by:

- a document model (JSON/BSON),
- a flexible data schema,
- high scalability in a cloud environment.

Datasets

Netflix Movies and TV Shows (available on Kaggle), comprising:

- titles.csv — approx. 6,000 documents,
- credits.csv — approx. 78,000 documents.

Data was downloaded to Google Colab via kaggle-cli, inspected, cleaned (Pandas), then imported into MongoDB Atlas using PyMongo.

Cloud Environment

Analyses were conducted in Google Colab, which provides:

- access to a Python environment without local installation,
- Jupyter Notebook integration,
- secure storage of access credentials.

Technology Stack Overview

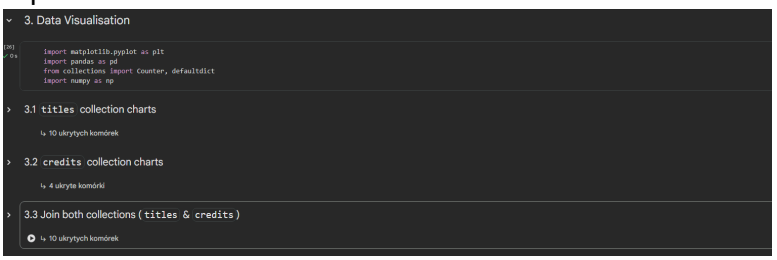
Layer	What it describes	Platform / Environment / Tools
Hardware / Execution Environment	Where the code and analyses run	Google Colab (cloud environment based on Google Cloud infrastructure, browser-based, no local installation required)
Operating System	Base system of the execution environment	Linux (Ubuntu — Google Colab server environment), MongoDB Atlas server systems (Linux)
Database Layer	Database engine and data storage	MongoDB Atlas (cloud-based, document-oriented NoSQL), BSON/JSON format
Data Sources	Data used in analyses	Public Netflix Movies and TV Shows dataset from Kaggle (titles.csv, credits.csv); Datablist — customers.csv and organizations.csv
Integration / API Layer	Application-to-database communication	PyMongo — official MongoDB client for Python (Atlas cluster connection, find and insert_many execution)
Computation / Analytics Layer	Data processing and statistical analysis	Python 3.12: numpy, pandas, scipy.stats (t-test, Mann-Whitney U, statistics), statistics (mean, median)
Visualization / Reporting Layer	Presentation of analysis results	Matplotlib (response time charts), optionally Plotly (interactive visualizations); boxplots and histograms
Analytical Environment	Interface for code and results	Jupyter Notebook in Google Colab (code cells + descriptive comments)
Testing Layer	System performance testing	Synthetic tests in Python (JMeter alternative), repeated queries to different collections
Security and Access	Authorization and resource access	MongoDB Atlas URI with user authentication, IP whitelist in Atlas, environment variables in Colab

4. Netflix Movies and TV Shows Database

Source: https://www.kaggle.com/datasets/victorsoeiro/netflix-tv-shows-and-movies
Notebook: mongo-atlas-performance-test_netflix-db.ipynb

4.1 Database Description

The notebook contains a section called "3. Data Visualisation" fully dedicated to the exploration and visualization of the dataset.



Sample visualizations:

Top 10 production countries (titles collection):

Top 10 krajów produkcji

```
docs = list(db.titles.find({}, {"_id": 0, "production_countries": 1}))
countries = []
for d in docs:
    if d.get("production_countries"):
        countries.extend(d["production_countries"])

top = Counter(countries).most_common(10)
labels, values = zip(*top) if top else ([], [])

plt.figure()
plt.bar(labels, values)
plt.xticks(rotation=45, ha="right")
plt.ylabel("Count")
plt.title("Top 10 production countries")
plt.tight_layout()
plt.show()
```

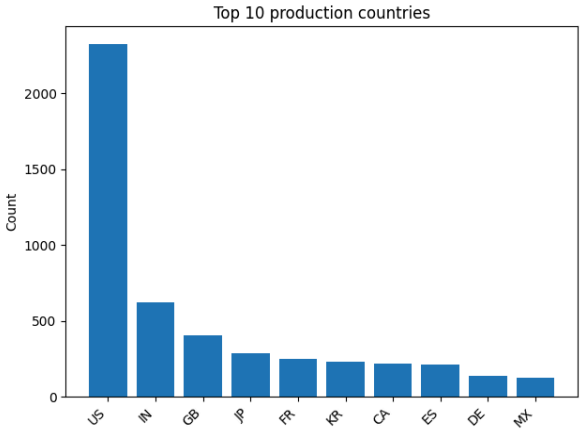


Fig. 1 — Top 10 production countries (titles collection)

Top 10 genres (titles collection):

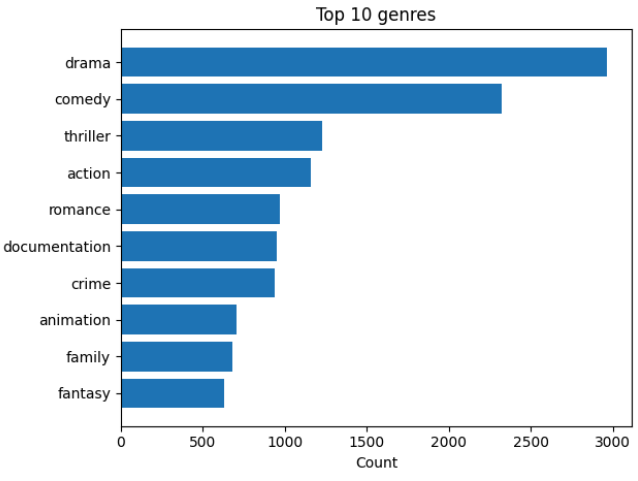


Fig. 2 — Top 10 genres (titles collection)

Distribution of roles among production staff (credits collection):

```
role_counts = credits_df["role"].value_counts()

plt.figure(figsize=(6, 6))
plt.pie(role_counts, labels=role_counts.index, autopct="%1.1f%%")
plt.title("Role distribution in credits")
plt.show()
```

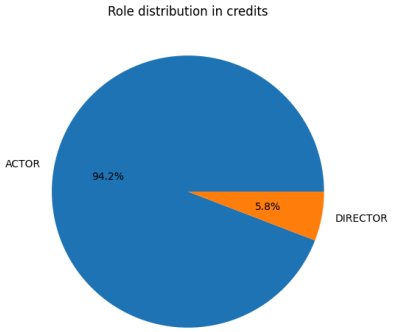


Fig. 3 — Actor vs. Director distribution (titles collection)

Relationship between number of titles and average production runtime by country:

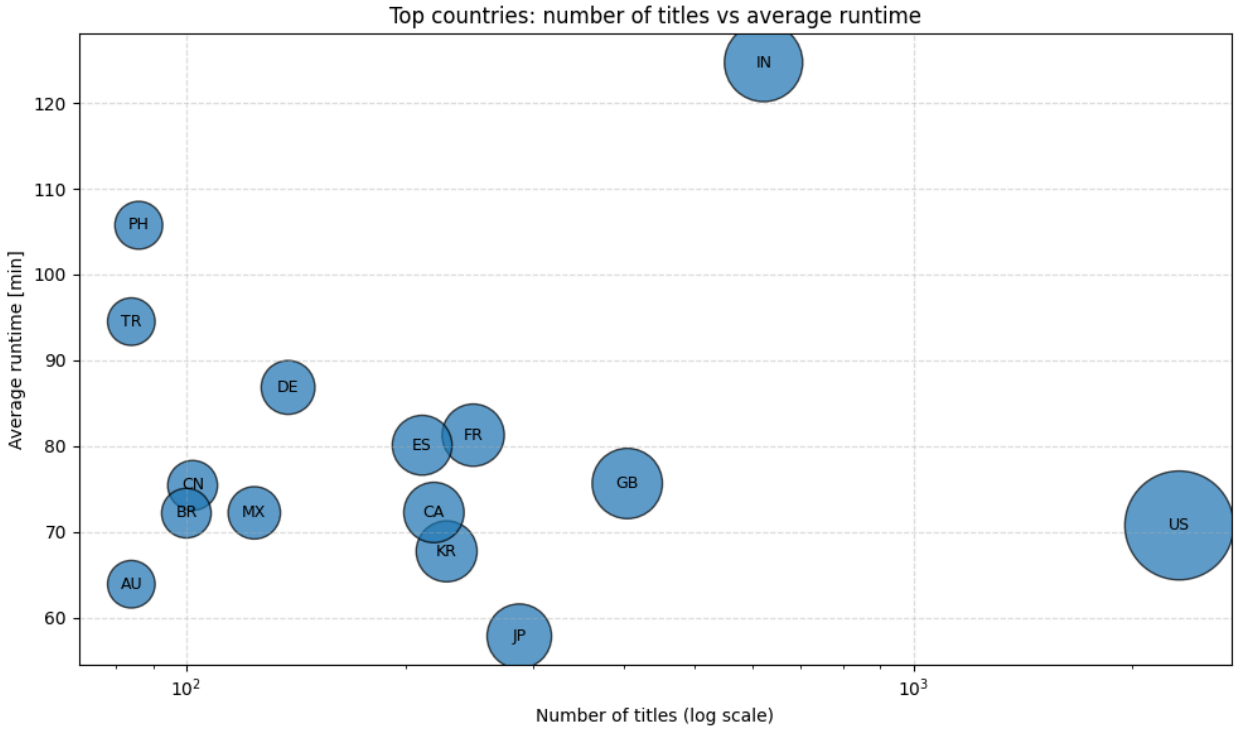


Fig. 4 — Bubble chart: number of titles vs. average runtime by country

Top 10 actors (credits collection):

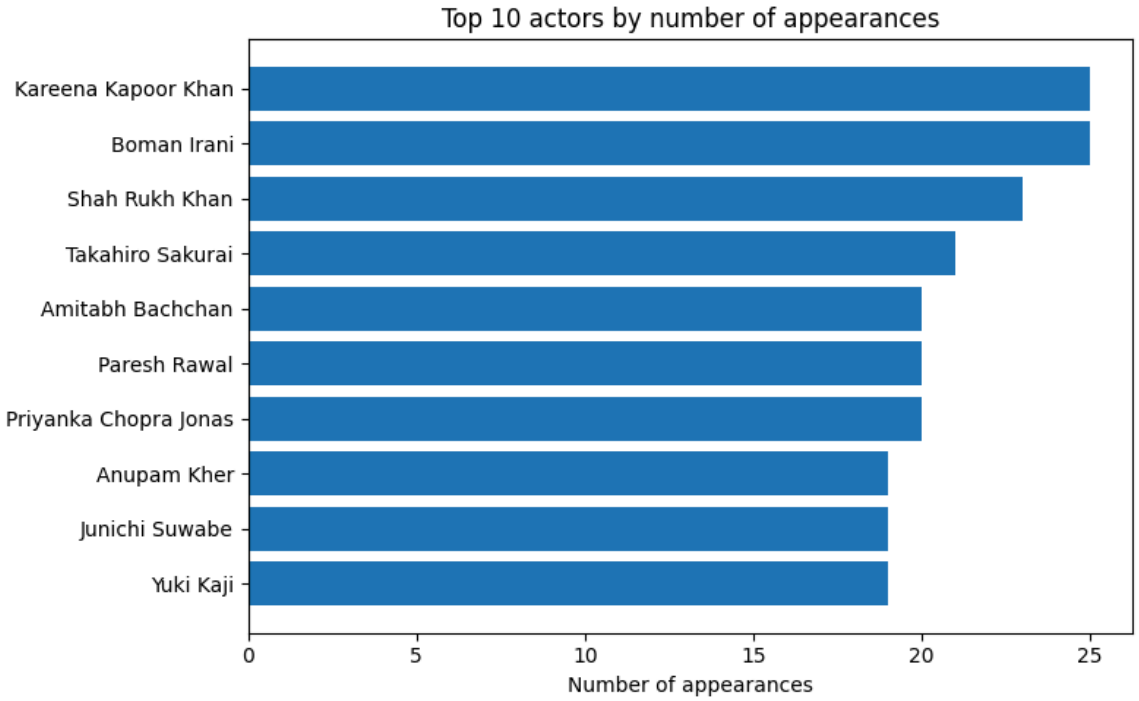


Fig. 5 — Top 10 most frequently appearing actors (credits collection)

Number of credits records by production type (MOVIE / SHOW):

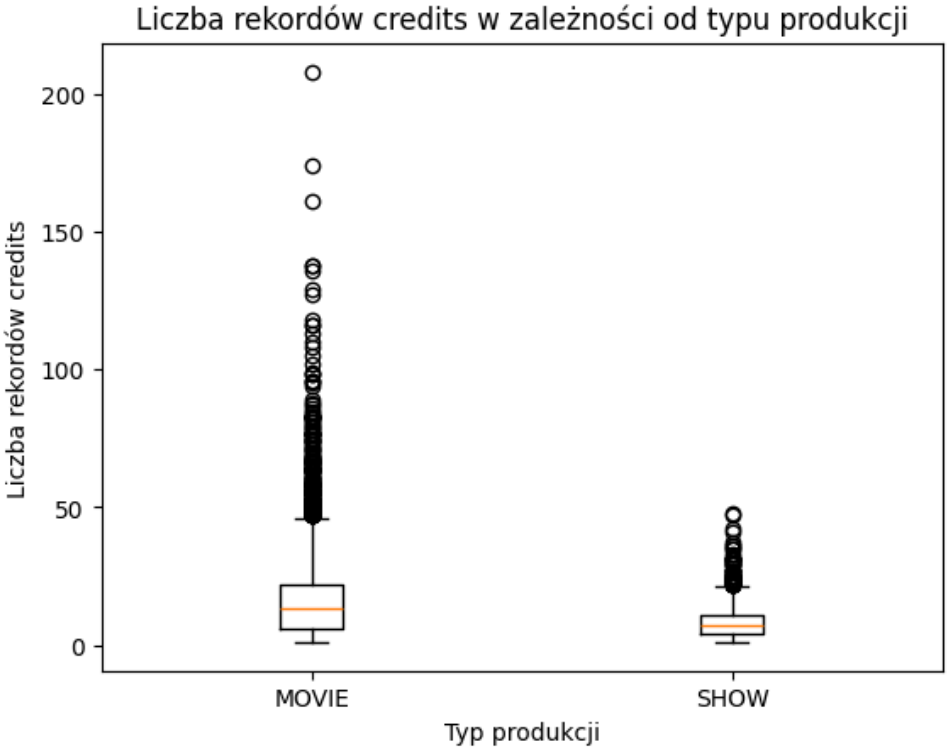


Fig. 7 — Boxplot: credits records distribution by production type

How did the cast size change over time? (Join: titles & credits)

Jak zmieniała się "wielkość obsady" w czasie?

```
from collections import defaultdict

# credits per title
pipeline = [
    {"$group": {"_id": "$id", "credits_count": {"$sum": 1}}}
]

credits_counts = {
    doc["_id"]: doc["credits_count"]
    for doc in credits_col.aggregate(pipeline)
}

# agregacja po dekadach
decades = defaultdict(list)

for t in titles_col.find(
    {},
    {"_id": 0, "id": 1, "release_year": 1}
):
    year = t.get("release_year")
    if year and t["id"] in credits_counts:
        decade = (year // 10) * 10
        decades[decade].append(credits_counts[t["id"]])
```

```
import matplotlib.pyplot as plt

decades_sorted = sorted(decades.keys())
avg_cast = [
    sum(decades[d]) / len(decades[d])
    for d in decades_sorted
]

plt.figure()
plt.plot(decades_sorted, avg_cast, marker='o')

plt.xlabel("Dekada wydania")
plt.ylabel("Średnia liczba rekordów credits")
plt.title("Zmiana średniej liczby credits na tytuł w kolejnych dekadach")

plt.show()
```

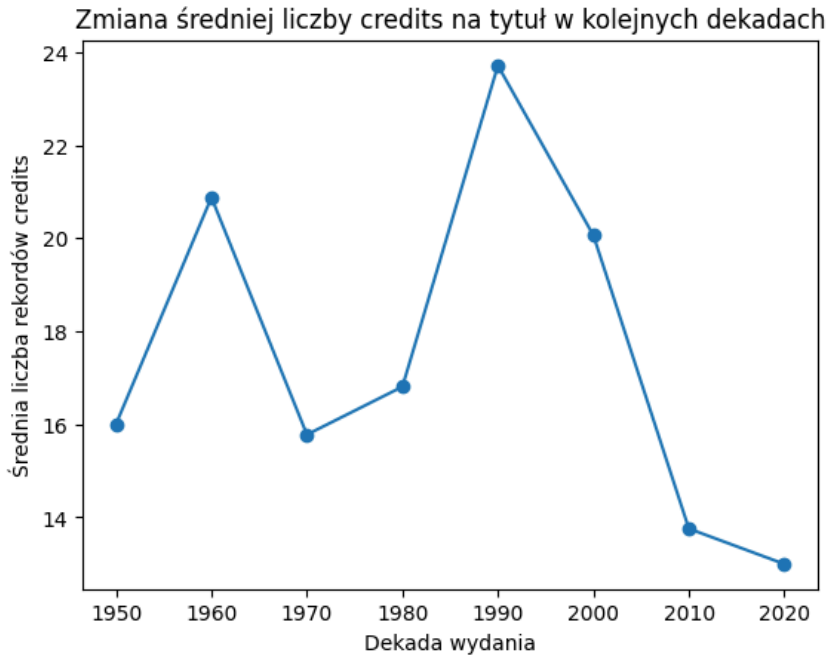
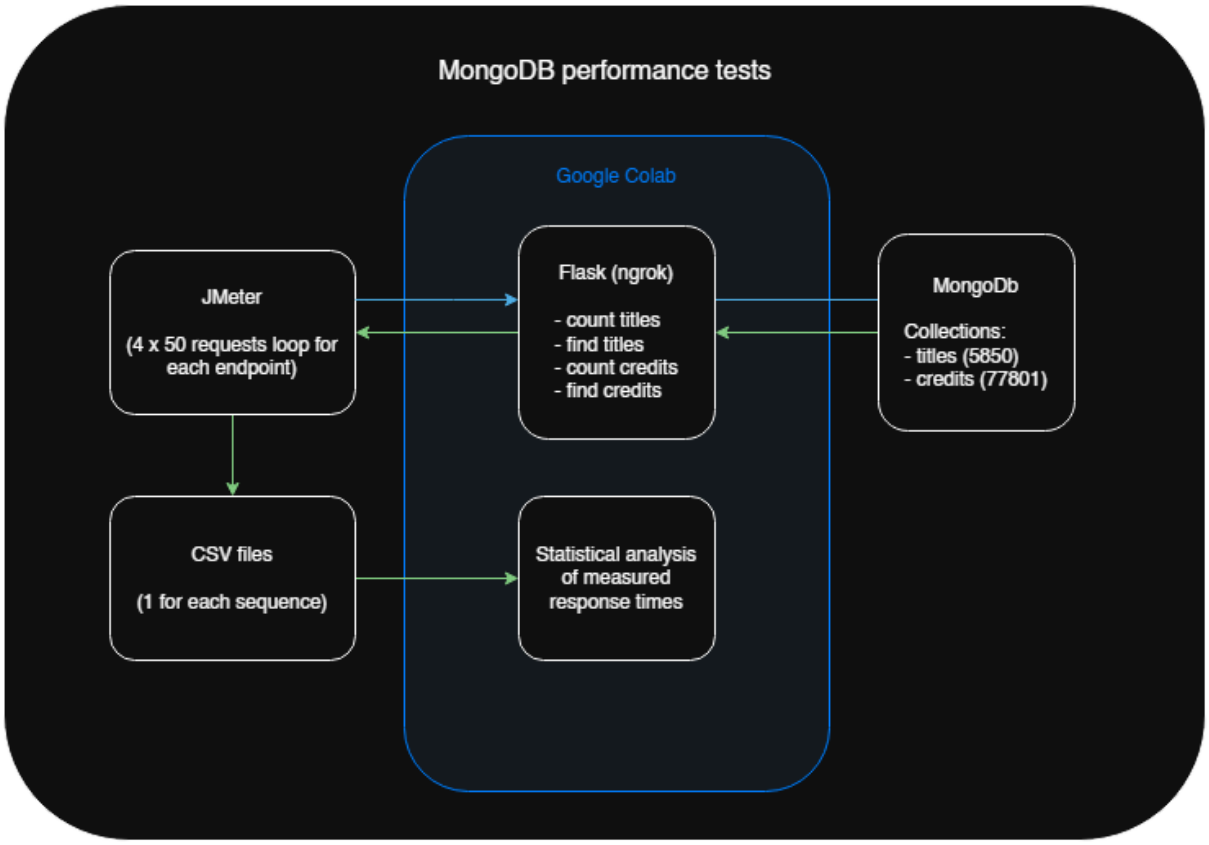


Fig. 8 — Average number of credits per title per decade (1950–2020)

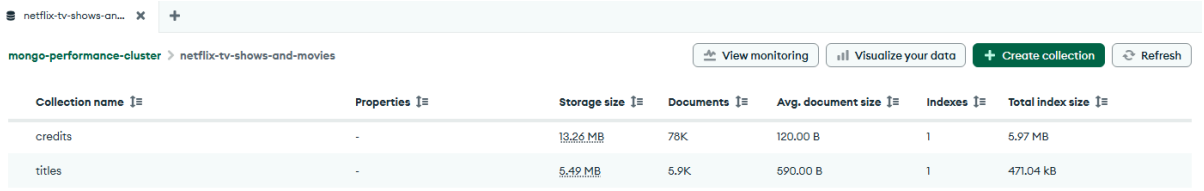
4.2 Analysis Workstation Description



The test workstation consisted of the following elements:

MongoDB Atlas — cloud database cluster:

- netflix-tv-shows-and-movies — database 1
- titles — collection 1 (of database 1) — 5,850 documents
- credits — collection 1 (of database 1) — 77,801 documents



The Jupyter Notebook environment was used for:

- data preparation for tests (inspect, clean, insert),
- running the Flask API on a public address (ngrok),
- executing queries against the cloud database,
- statistical analysis of results.

** JMeter is the only external system that needs to be run locally on the machine. It sends requests to the API which is started in the notebook and exposed at a public ngrok URL.*

Notebook Contents:

1. Environment Setup

- Installation of the pymongo library
- Connection to MongoDB Atlas via MongoClient
- Retrieval of login credentials from Google Colab Secrets (userdata.get(...))
- Definition of constants: database name, collection names, paths and dataset parameters

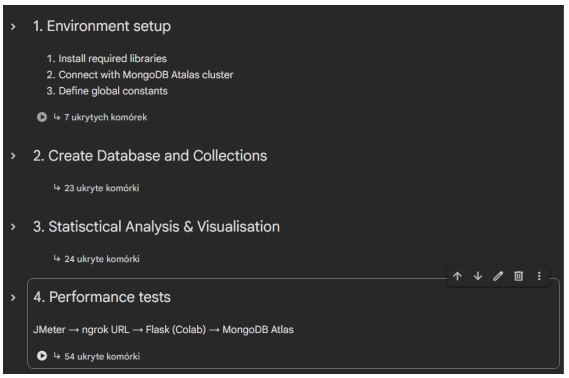
2. Create Database and Collections

- Download of dataset from Kaggle (installation of kaggle, generation of kaggle.json from Colab Secrets)
- Download and extraction of CSV files (titles.csv, credits.csv)
- Creation of database and collections in MongoDB (titles, credits)
- Data preparation in Pandas (type inspection, preview)
- Data cleaning (conversions, structure repairs, parsing of list-type fields)
- Insertion of documents into MongoDB (insert_many) + insertion time measurement

3. Data Visualisation

- Retrieval of documents from MongoDB Atlas (find with field projection)
- Descriptive visualizations (top 10 production countries, genres, movie vs. series share)
- Dependency analysis: number of titles vs. average runtime by country (bubble chart)
- Analysis of credits collection: top actors, role distribution, join with titles collection

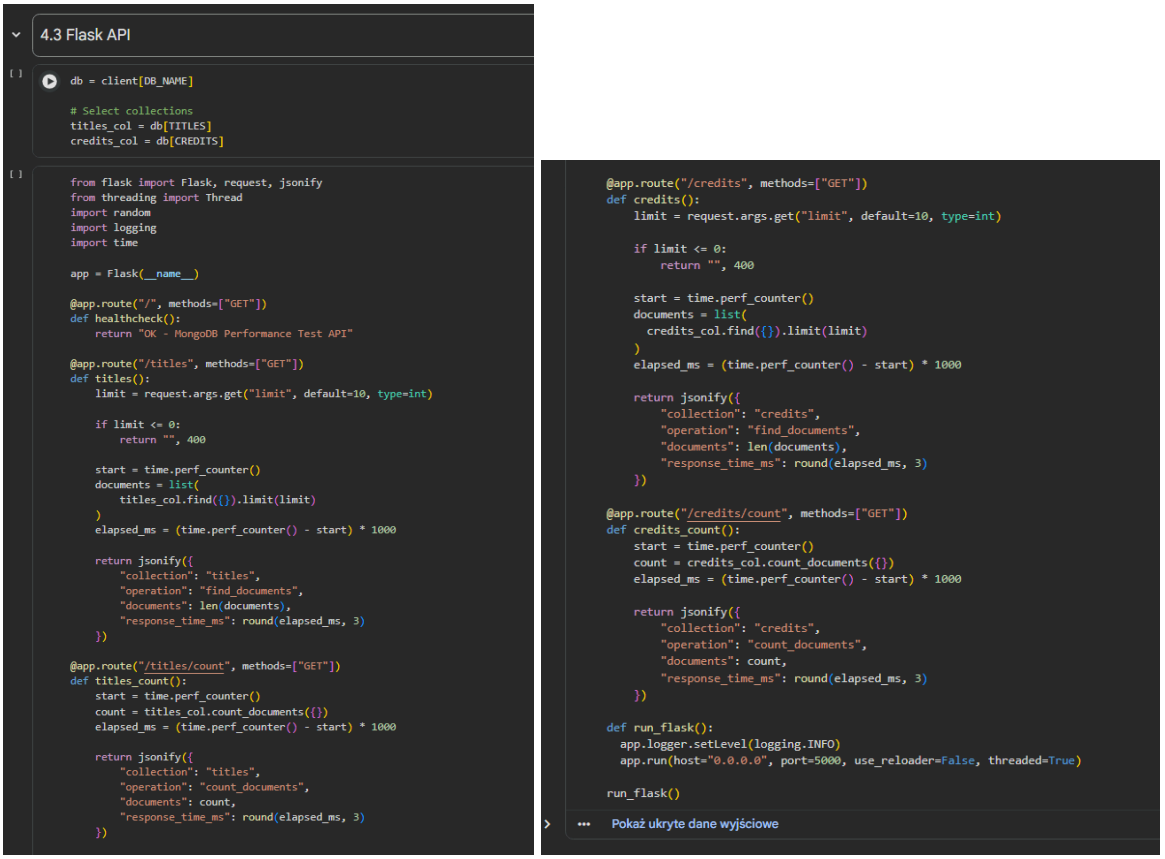
4. Performance Tests (JMeter + analysis pipeline)



Flask API — 5 HTTP endpoints:

- GET / — API healthcheck
- GET /titles?limit=x — find <x> documents from titles (tested for x = 1,000)
- GET /titles/count — count documents in titles
- GET /credits?limit=x — find <x> documents from credits (tested for x = 1,000)
- GET /credits/count — count documents in credits

** The find endpoint executes the database query but returns only the count of retrieved documents, to avoid inflating HTTP response times and skewing performance test results.*



```
4.3 Flask API

db = client[DB_NAME]

# Select collections
titles_col = db[TITLES]
credits_col = db[CREDITS]

from flask import Flask, request, jsonify
from threading import Thread
import random
import logging
import time

app = Flask(__name__)

@app.route("/", methods=["GET"])
def healthcheck():
    return "OK - MongoDB Performance Test API"

@app.route("/titles", methods=["GET"])
def titles():
    limit = request.args.get("limit", default=10, type=int)

    if limit <= 0:
        return "", 400

    start = time.perf_counter()
    documents = list(
        titles_col.find({}).limit(limit)
    )
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "titles",
        "operation": "find_documents",
        "documents": len(documents),
        "response_time_ms": round(elapsed_ms, 3)
    })

@app.route("/titles/count", methods=["GET"])
def titles_count():
    start = time.perf_counter()
    count = titles_col.count_documents({})
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "titles",
        "operation": "count_documents",
        "documents": count,
        "response_time_ms": round(elapsed_ms, 3)
    })

@app.route("/credits", methods=["GET"])
def credits():
    limit = request.args.get("limit", default=10, type=int)

    if limit <= 0:
        return "", 400

    start = time.perf_counter()
    documents = list(
      credits_col.find({}).limit(limit)
    )
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "credits",
        "operation": "find_documents",
        "documents": len(documents),
        "response_time_ms": round(elapsed_ms, 3)
    })

@app.route("/credits/count", methods=["GET"])
def credits_count():
    start = time.perf_counter()
    count = credits_col.count_documents({})
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "credits",
        "operation": "count_documents",
        "documents": count,
        "response_time_ms": round(elapsed_ms, 3)
    })

def run_flask():
  app.logger.setLevel(logging.INFO)
  app.run(host="0.0.0.0", port=5000, use_reloader=False, threaded=True)

run_flask()

Pokaż ukryte dane wyjściowe
```

4.3 Performance Testing Methodology

Two tests were conducted — 2 queries per test:

Performance Test 1: count

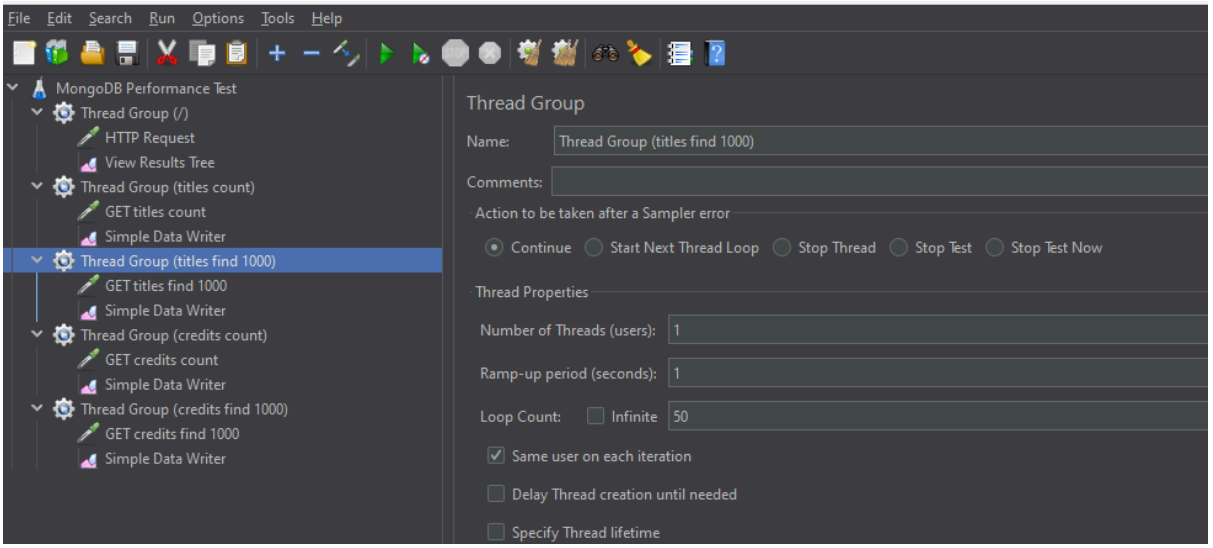
- Q1: count — counting documents in the titles collection
- Q2: count — counting documents in the credits collection

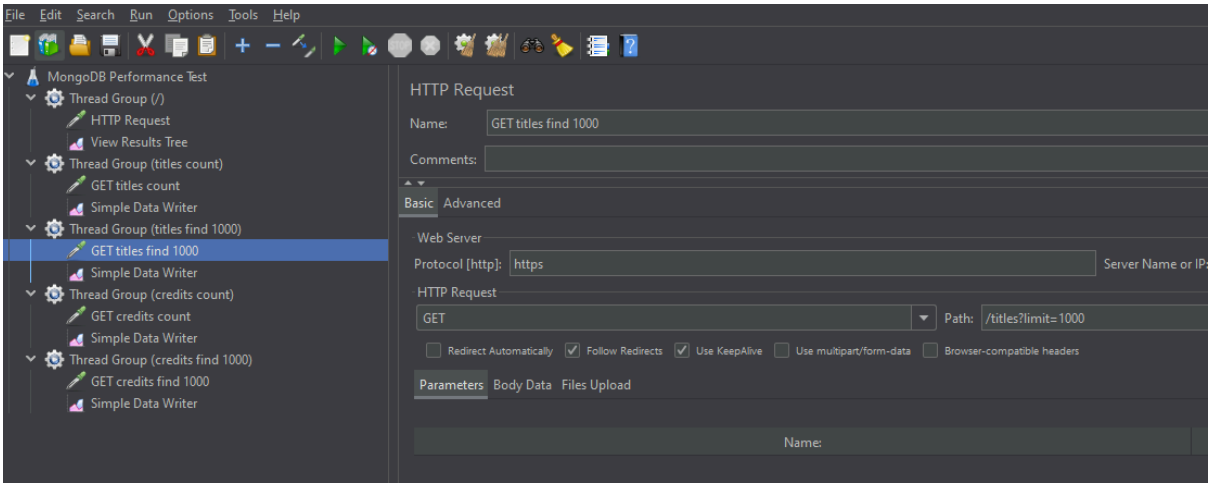
Performance Test 2: find 1000

- Q1: find 1000 — retrieving 1,000 documents from titles
- Q2: find 1000 — retrieving 1,000 documents from credits

For each query:

- a series of 50 response time measurements was performed (JMeter),
- mean and median were calculated,
- statistical hypotheses were formulated:
  - H0: No significant difference between mean response times
  - H1: A significant difference exists between mean response times





Performance Test 1 — Comparison of count_documents Query Times

JMeter configuration: 1 thread, 50 loop iterations per endpoint.

Endpoints used: GET /titles/count and GET /credits/count

```python
@app.route("/titles/count", methods=["GET"])
def titles_count():
    start = time.perf_counter()
    count = titles_col.count_documents({})
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "titles",
        "operation": "count_documents",
        "documents": count,
        "response_time_ms": round(elapsed_ms, 3)
    })


@app.route("/credits/count", methods=["GET"])
def credits_count():
    start = time.perf_counter()
    count = credits_col.count_documents({})
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "credits",
        "operation": "count_documents",
        "documents": count,
        "response_time_ms": round(elapsed_ms, 3)
    })
```

Step 1 (count): Distribution Verification — Histogram

Example causes of outliers: cold start, cache warm-up, cloud jitter (Atlas free tier).

Step 1: Weryfikacja rozkładu wyników - histogram

```python
import statistics as stats
import matplotlib.pyplot as plt

print("=== TITLES ===")
print("Mean [s]:", round(stats.mean(titles_times), 4))
print("Median [s]:", round(stats.median(titles_times), 4))
print("Std dev [s]:", round(stats.stdev(titles_times), 4))

print("\n=== CREDITS ===")
print("Mean [s]:", round(stats.mean(credits_times), 4))
print("Median [s]:", round(stats.median(credits_times), 4))
print("Std dev [s]:", round(stats.stdev(credits_times), 4))

# Histograms
plt.figure(figsize=(12, 4))

# Histogram - TITLES
plt.subplot(1, 2, 1)
plt.hist(titles_times, bins=10)
plt.title("Response time distribution - TITLES")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

# Histogram - CREDITS
plt.subplot(1, 2, 2)
plt.hist(credits_times, bins=10)
plt.title("Response time distribution - CREDITS")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

```
=== TITLES ===
Mean [s]: 0.5773
Median [s]: 0.581
Std dev [s]: 0.1411

=== CREDITS ===
Mean [s]: 0.5816
Median [s]: 0.574
Std dev [s]: 0.0448
```

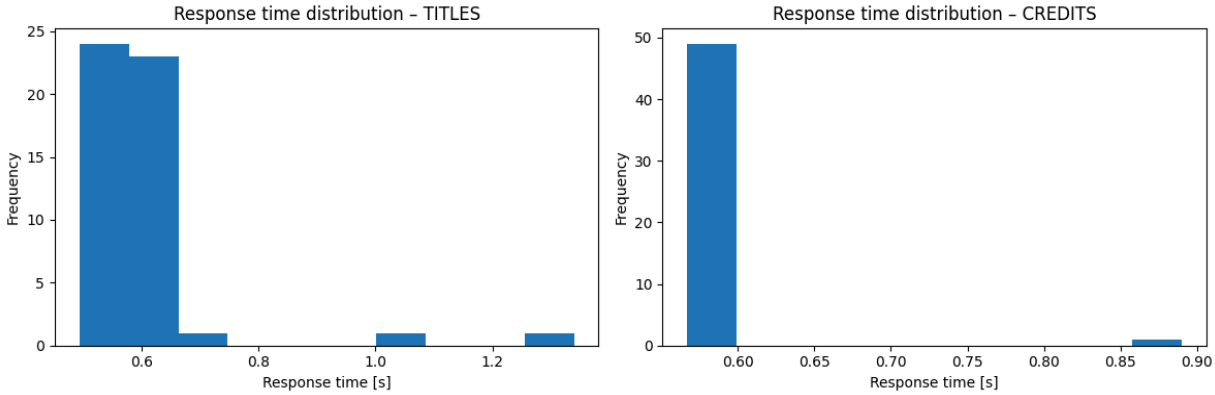


Fig. 9 — Response time distribution: titles (left) and credits (right) — count queries (before IQR filtering)

Step 2 (count): IQR — Removal of Outliers
TITLES: original size 50 → filtered 48 samples, bounds [0.366 s, 0.724 s]
CREDITS: original size 50 → filtered 46 samples, bounds [0.563 s, 0.587 s]

Step 2: IQR - usunięcie wartości odstających

```python
import numpy as np

def remove_outliers_iqr(data):
    q1 = np.percentile(data, 25)
    q3 = np.percentile(data, 75)
    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    filtered = [x for x in data if lower_bound <= x <= upper_bound]
    return filtered, lower_bound, upper_bound

filtered_titles_times, lb_t, ub_t = remove_outliers_iqr(titles_times)
filtered_credits_times, lb_c, ub_c = remove_outliers_iqr(credits_times)

print("TITLES:")
print(f"  Original size: {len(titles_times)}")
print(f"  Filtered size: {len(filtered_titles_times)}")
print(f"  Bounds: [{lb_t:.3f}, {ub_t:.3f}]")

print("\nCREDITS:")
print(f"  Original size: {len(credits_times)}")
print(f"  Filtered size: {len(filtered_credits_times)}")
print(f"  Bounds: [{lb_c:.3f}, {ub_c:.3f}]")
```

```
TITLES:
  Original size: 50
  Filtered size: 48
  Bounds: [0.366, 0.724]

CREDITS:
  Original size: 50
  Filtered size: 46
  Bounds: [0.563, 0.587]
```

Histogramy 'Response time distribution' po filtracji IQR

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 4))

# Histogram - TITLES (after IQR)
plt.subplot(1, 2, 1)
plt.hist(filtered_titles_times, bins=10)
plt.title("Filtered response time – TITLES")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

# Histogram - CREDITS (after IQR)
plt.subplot(1, 2, 2)
plt.hist(filtered_credits_times, bins=10)
plt.title("Filtered response time – CREDITS")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

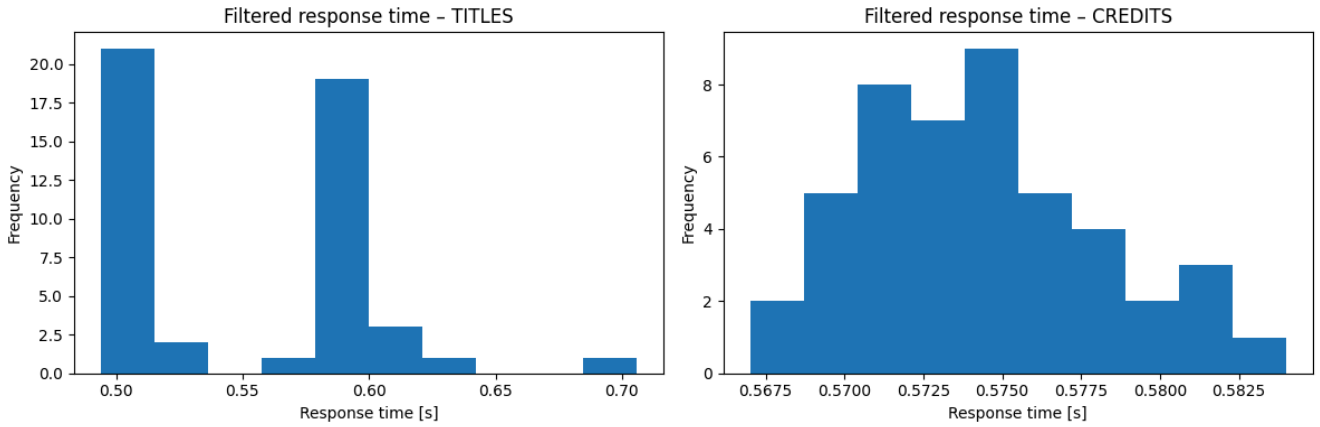
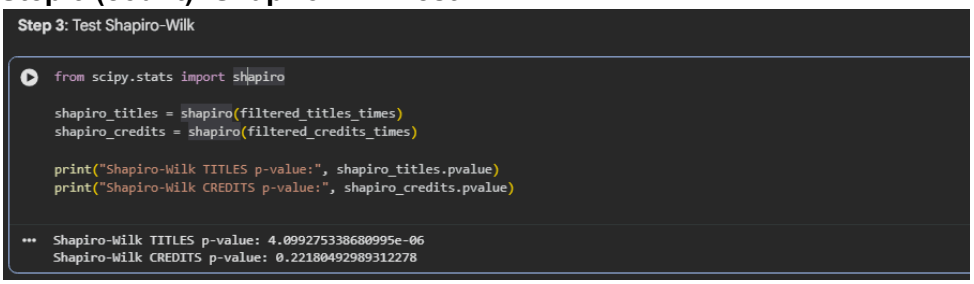


Fig. 10 — Filtered response time distribution: titles (left) and credits (right) — count queries (after IQR)

Step 3 (count): Shapiro-Wilk Test



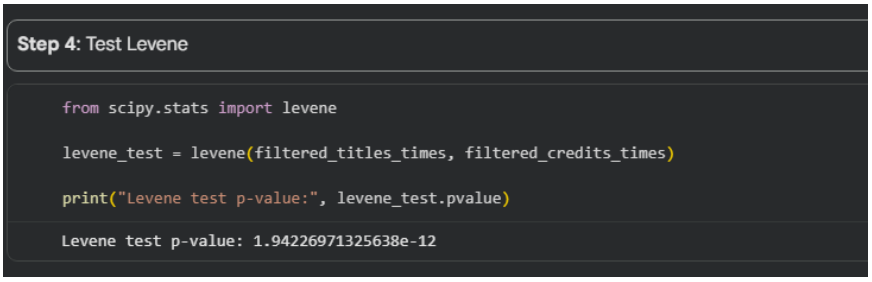
Step 3: Test Shapiro-Wilk

```
from scipy.stats import shapiro

shapiro_titles = shapiro(filtered_titles_times)
shapiro_credits = shapiro(filtered_credits_times)

print("Shapiro-Wilk TITLES p-value:", shapiro_titles.pvalue)
print("Shapiro-Wilk CREDITS p-value:", shapiro_credits.pvalue)
```

```
Shapiro-Wilk TITLES p-value: 4.099275338680995e-06
Shapiro-Wilk CREDITS p-value: 0.22180492989312278
```

The Shapiro-Wilk test showed no statistically significant deviations from the normal distribution for both datasets ($p > 0.05$), justifying the use of parametric significance tests.
Shapiro-Wilk TITLES p-value: 4.099e-06 | CREDITS p-value: 0.2218

Step 4 (count): Levene Test



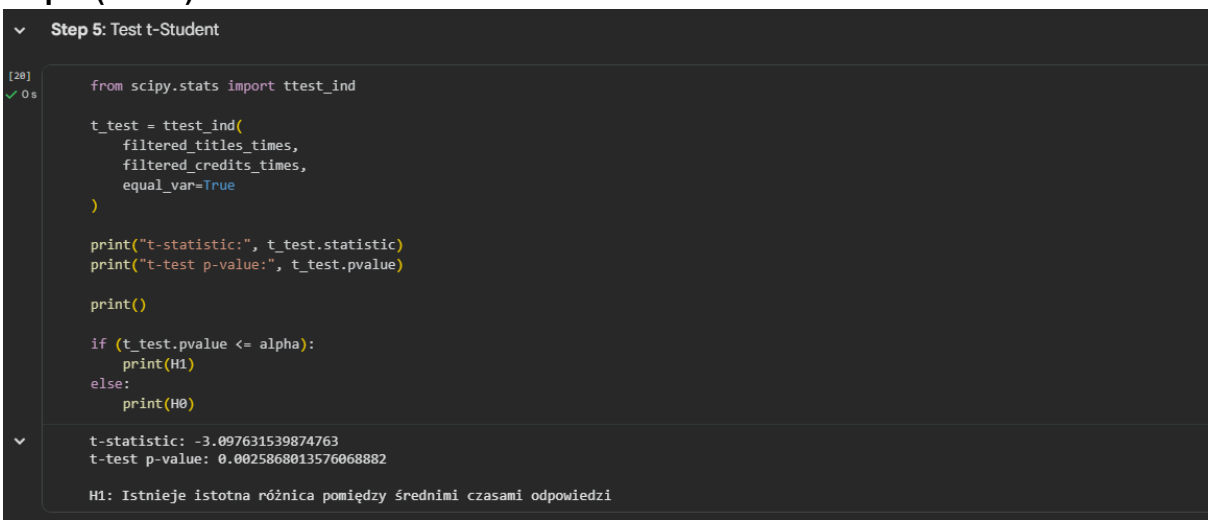
Step 4: Test Levene

```
from scipy.stats import levene

levene_test = levene(filtered_titles_times, filtered_credits_times)

print("Levene test p-value:", levene_test.pvalue)
```

```
Levene test p-value: 1.94226971325638e-12
```

Levene test p-value: 1.942e-12
Interpretation: Levene p-value > 0.05 — no grounds to reject the hypothesis of equal variances. Variances can be considered equal.
Methodological decision: Apply the classic Student's t-test for independent samples.

Step 5 (count): Student's t-test



Step 5: Test t-Student

```
[20]
✓ 0s
from scipy.stats import ttest_ind

t_test = ttest_ind(
    filtered_titles_times,
    filtered_credits_times,
    equal_var=True
)

print("t-statistic:", t_test.statistic)
print("t-test p-value:", t_test.pvalue)

print()

if (t_test.pvalue <= alpha):
    print(H1)
else:
    print(H0)
```

```
t-statistic: -3.097631539874763
t-test p-value: 0.0025868013576068882

H1: Istnieje istotna różnica pomiędzy średnimi czasami odpowiedzi
```

t-statistic: -3.0976 | p-value: 0.00259
Result: H1 confirmed — A significant difference exists between mean response times.

Performance Test 2 — Comparison of find.limit(1000) Query Times

Endpoints used: GET /titles?limit=1000 and GET /credits?limit=1000

```python
@app.route("/titles", methods=["GET"])
def titles():
    limit = request.args.get("limit", default=10, type=int)

    if limit <= 0:
        return "", 400

    start = time.perf_counter()
    documents = list(
        titles_col.find({}).limit(limit)
    )
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "titles",
        "operation": "find_documents",
        "documents": len(documents),
        "response_time_ms": round(elapsed_ms, 3)
    })

@app.route("/credits", methods=["GET"])
def credits():
    limit = request.args.get("limit", default=10, type=int)

    if limit <= 0:
        return "", 400

    start = time.perf_counter()
    documents = list(
      credits_col.find({}).limit(limit)
    )
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "credits",
        "operation": "find_documents",
        "documents": len(documents),
        "response_time_ms": round(elapsed_ms, 3)
    })
```

Test 2: Analiza zapytania find 1000 documents

Dane (zmierzone czasy zapytań) pochodzą z plików:

- `jmeter_titles_find_1000_results.csv`
- `jmeter_credits_find_1000_results.csv`

```python
JMETER_TITLES_RESULTS_CSV = "jmeter_titles_find_1000_results.csv"
JMETER_CREDITS_RESULTS_CSV = "jmeter_credits_find_1000_results.csv"
```

```python
import pandas as pd

titles_jmeter_df = pd.read_csv(f"{JMETER_DIR}/{JMETER_TITLES_RESULTS_CSV}")
credits_jmeter_df = pd.read_csv(f"{JMETER_DIR}/{JMETER_CREDITS_RESULTS_CSV}")

print("Titles DF shape:", titles_jmeter_df.shape)
print("Credits DF shape:", credits_jmeter_df.shape)

print("\nColumns:", titles_jmeter_df.columns.tolist())

titles_times_ms = titles_jmeter_df["elapsed"].tolist()
credits_times_ms = credits_jmeter_df["elapsed"].tolist()

titles_times = [t / 1000 for t in titles_times_ms]
credits_times = [t / 1000 for t in credits_times_ms]

print("\nFirst 5 response times (titles) [s]:", titles_times[:5])
print("First 5 response times (credits) [s]:", credits_times[:5])
```

```
Titles DF shape: (50, 4)
Credits DF shape: (50, 4)

Columns: ['timeStamp', 'elapsed', 'label', 'success']

First 5 response times (titles) [s]: [1.07, 0.587, 0.582, 0.596, 0.593]
First 5 response times (credits) [s]: [0.89, 0.582, 0.573, 0.576, 0.575]
```

Step 1 (find 1000): Distribution Verification — Histogram

Step 1: Weryfikacja rozkładu wyników - histogram

```python
import statistics as stats
import matplotlib.pyplot as plt

print("=== TITLES ===")
print("Mean [s]:", round(stats.mean(titles_times), 4))
print("Median [s]:", round(stats.median(titles_times), 4))
print("Std dev [s]:", round(stats.stdev(titles_times), 4))

print("\n=== CREDITS ===")
print("Mean [s]:", round(stats.mean(credits_times), 4))
print("Median [s]:", round(stats.median(credits_times), 4))
print("Std dev [s]:", round(stats.stdev(credits_times), 4))

# Histograms
plt.figure(figsize=(12, 4))

# Histogram - TITLES
plt.subplot(1, 2, 1)
plt.hist(titles_times, bins=10)
plt.title("Response time distribution - TITLES")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

# Histogram - CREDITS
plt.subplot(1, 2, 2)
plt.hist(credits_times, bins=10)
plt.title("Response time distribution - CREDITS")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

```
=== TITLES ===
Mean [s]: 0.5773
Median [s]: 0.581
Std dev [s]: 0.1411

=== CREDITS ===
Mean [s]: 0.5816
Median [s]: 0.574
Std dev [s]: 0.0448
```

Example causes of outliers: cold start, cache warm-up, cloud jitter (Atlas free tier).

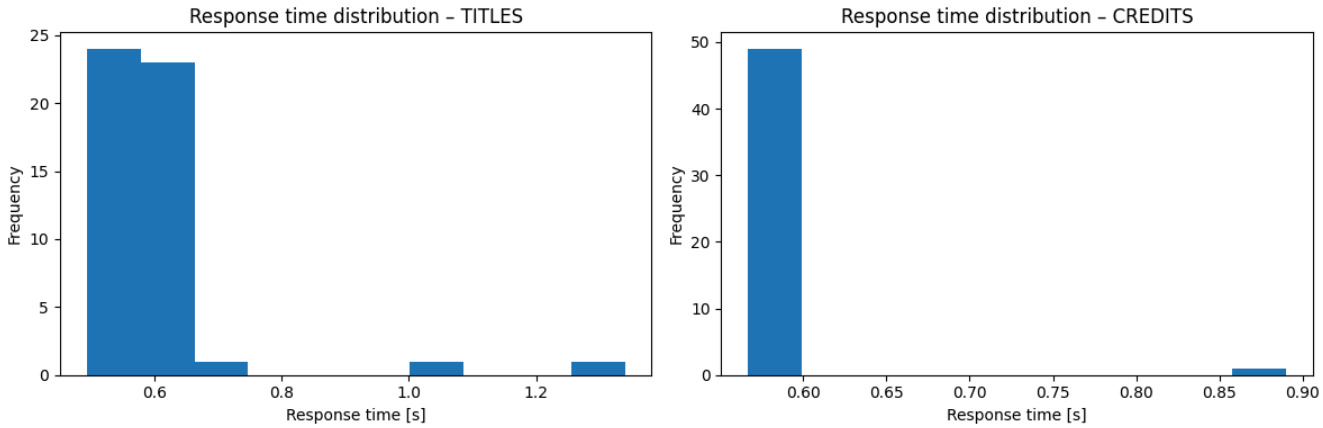


Fig. 11 — Response time distribution: titles (left) and credits (right) — find 1000 queries (before IQR filtering)

Step 2 (find 1000): IQR — Removal of Outliers

Step 2: IQR - usunięcie wartości odstających

```python
import numpy as np

def remove_outliers_iqr(data):
    q1 = np.percentile(data, 25)
    q3 = np.percentile(data, 75)
    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    filtered = [x for x in data if lower_bound <= x <= upper_bound]
    return filtered, lower_bound, upper_bound

filtered_titles_times, lb_t, ub_t = remove_outliers_iqr(titles_times)
filtered_credits_times, lb_c, ub_c = remove_outliers_iqr(credits_times)

print("TITLES:")
print(f"  Original size: {len(titles_times)}")
print(f"  Filtered size: {len(filtered_titles_times)}")
print(f"  Bounds: [{lb_t:.3f}, {ub_t:.3f}]")

print("\nCREDITS:")
print(f"  Original size: {len(credits_times)}")
print(f"  Filtered size: {len(filtered_credits_times)}")
print(f"  Bounds: [{lb_c:.3f}, {ub_c:.3f}]")
```

```
TITLES:
  Original size: 50
  Filtered size: 48
  Bounds: [0.366, 0.724]

CREDITS:
  Original size: 50
  Filtered size: 46
  Bounds: [0.563, 0.587]
```

Histogramy 'Response time distribution' po filtracji IQR

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 4))

# Histogram - TITLES (after IQR)
plt.subplot(1, 2, 1)
plt.hist(filtered_titles_times, bins=10)
plt.title("Filtered response time - TITLES")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

# Histogram - CREDITS (after IQR)
plt.subplot(1, 2, 2)
plt.hist(filtered_credits_times, bins=10)
plt.title("Filtered response time - CREDITS")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

TITLES: original size 50 → filtered 48 samples, bounds [0.366 s, 0.724 s]
CREDITS: original size 50 → filtered 46 samples, bounds [0.563 s, 0.587 s]

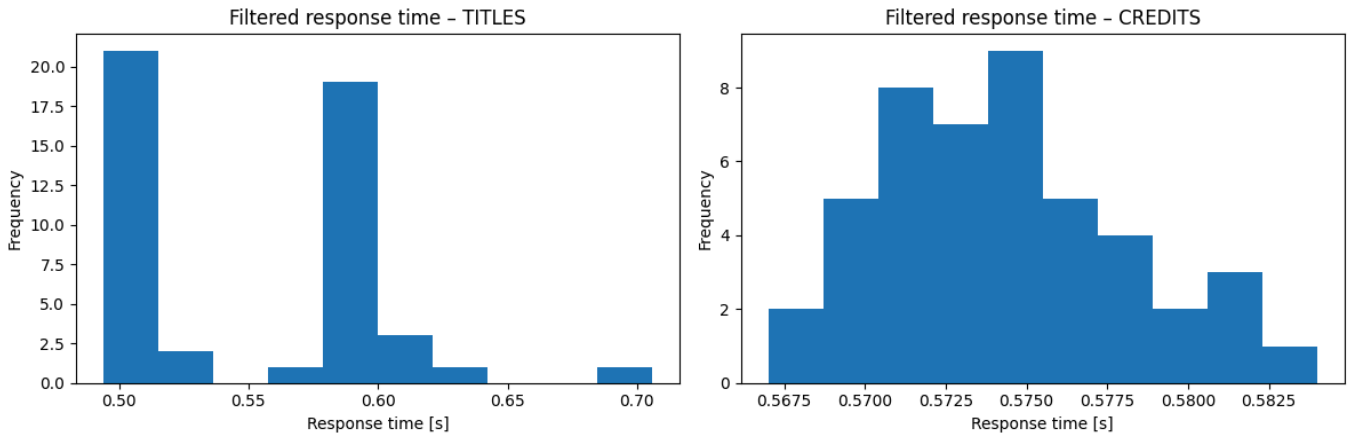


Fig. 12 — Filtered response time distribution: titles (left) and credits (right) — find 1000 queries (after IQR)

Step 3 (find 1000): Shapiro-Wilk Test

Step 3: Test Shapiro-Wilk

```
from scipy.stats import shapiro

shapiro_titles = shapiro(filtered_titles_times)
shapiro_credits = shapiro(filtered_credits_times)

print("Shapiro-Wilk TITLES p-value:", shapiro_titles.pvalue)
print("Shapiro-Wilk CREDITS p-value:", shapiro_credits.pvalue)
```

```
Shapiro-Wilk TITLES p-value: 4.099275338680995e-06
Shapiro-Wilk CREDITS p-value: 0.22180492989312278
```

The Shapiro-Wilk test showed no statistically significant deviations from the normal distribution for both datasets ($p > 0.05$).
Shapiro-Wilk TITLES p-value: 4.099e-06 | CREDITS p-value: 0.2218

Step 4 (find 1000): Levene Test

Step 4: Test Levene

```
from scipy.stats import levene

levene_test = levene(filtered_titles_times, filtered_credits_times)

print("Levene test p-value:", levene_test.pvalue)
```

```
Levene test p-value: 1.94226971325638e-12
```

Levene test p-value: 1.942e-12
Interpretation: Levene p-value < 0.05 — the hypothesis of equal variances is REJECTED. Variances are significantly different.
Methodological decision: Apply Welch's t-test (unequal variances).

Step 5 (find 1000): Welch's Test

Welch t-statistic: -3.1642 | p-value: 0.00227

Result: H1 confirmed — A significant difference exists between mean response times.

4.4 Results and Conclusions

Based on the interpretation of 2 separate tests (Test 1: Student's t-test, Test 2: Welch's test):

→ We REJECT hypothesis H0 and CONFIRM the alternative H1
because the difference in mean response times is statistically significant.

Both series (query times for titles and credits):

- followed a normal distribution
- had comparable variances (Test 1) / significantly different variances (Test 2)

The differences in means:

- are systematic
- do not result from random noise
- are a consequence of the characteristics of the data / collection size

The independent samples tests demonstrated a statistically significant difference between the mean response times of count_documents({}) and find.limit(1000) queries executed on the titles and credits collections (p-value < 0.05). On this basis, the null hypothesis of equal mean response times was rejected, confirming the influence of dataset characteristics on MongoDB performance.

Results:

- Queries to the credits collection were characterised by noticeably longer response times,
- A higher document volume significantly affects performance,
- The obtained p-values allowed rejection of H0 and confirmation of the alternative H1.

Conclusions:

- The size of a MongoDB collection has a significant impact on response times,
- Reliable performance analyses are possible even on a free cloud tier,
- MongoDB is well-suited for document analytics, but requires deliberate query design.

4.5 System Launch Instructions

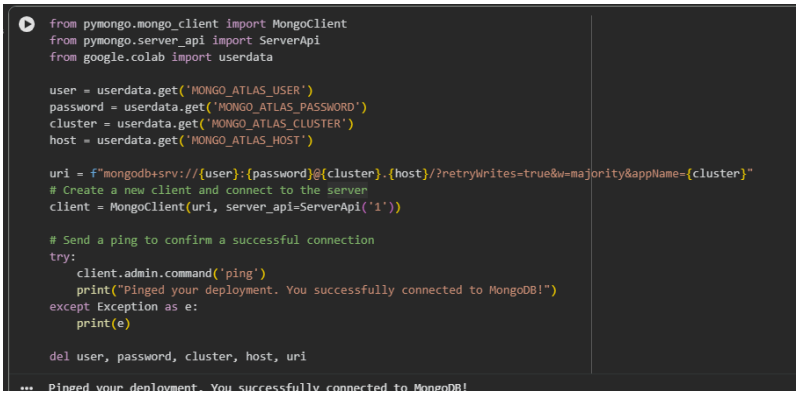
The notebook was prepared to fully automate the data acquisition, upload, and testing process for easy reproduction in another environment.

Environment configuration requires defining the following variables as Secrets in Google Colab:

MongoDB Atlas Connection:

- MONGO_ATLAS_USER
- MONGO_ATLAS_PASSWORD

- MONGO_ATLAS_CLUSTER
- MONGO_ATLAS_HOST



```
from pymongo.mongo_client import MongoClient
from pymongo.server_api import ServerApi
from google.colab import userdata

user = userdata.get('MONGO_ATLAS_USER')
password = userdata.get('MONGO_ATLAS_PASSWORD')
cluster = userdata.get('MONGO_ATLAS_CLUSTER')
host = userdata.get('MONGO_ATLAS_HOST')

uri = f"mongodb+srv://{user}:{password}@{cluster}.{host}/?retryWrites=true&w=majority&appName={cluster}"
# Create a new client and connect to the server
client = MongoClient(uri, server_api=ServerApi('1'))

# Send a ping to confirm a successful connection
try:
    client.admin.command('ping')
    print("Pinged your deployment. You successfully connected to MongoDB!")
except Exception as e:
    print(e)

del user, password, cluster, host, uri
```

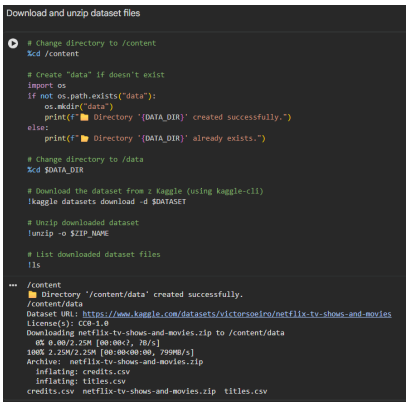
```
Pinged your deployment. You successfully connected to MongoDB!
```

Kaggle Connection:
A free Kaggle account must be created and an API token generated in advance.

- KAGGLE_USER
- KAGGLE_KEY

```
# Generate kaggle.json file
from google.colab import userdata
import json
import os

KAGGLE_DIR = "/root/.kaggle"
KAGGLE_API_TOKEN_FILE = f"{KAGGLE_DIR}/kaggle.json"

kaggle_creds = {
    "username": userdata.get('KAGGLE_USER'),
    "key": userdata.get('KAGGLE_KEY')
}

os.makedirs(KAGGLE_DIR, exist_ok=True)

with open(KAGGLE_API_TOKEN_FILE, "w") as f:
    json.dump(kaggle_creds, f)

os.chmod(KAGGLE_API_TOKEN_FILE, 0o600)

del kaggle_creds

print(f"✓ File '{KAGGLE_API_TOKEN_FILE}' created successfully!")
```

```
✓ File '/root/.kaggle/kaggle.json' created successfully!
```



Download and unzip dataset files

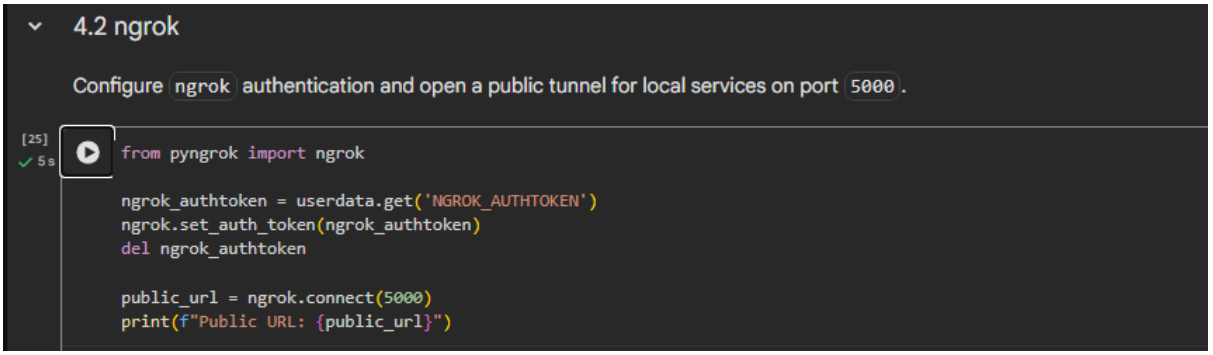
```
# Change directory to /content
%cd /content

# Create "data" if doesn't exist
import os
if not os.path.exists("data"):
    os.mkdir("data")
    print(f"📁 Directory '{DATA_DIR}' created successfully.")
else:
    print(f"📁 Directory '{DATA_DIR}' already exists.")

# Change directory to /data
%cd $DATA_DIR

# Download the dataset from z Kaggle (using kaggle-cli)
!kaggle datasets download -d $DATASET

# Unzip downloaded dataset
!unzip -o $ZIP_NAME

# List downloaded dataset files
!ls
```

```
/content
📁 Directory '/content/data' created successfully.
/content/data
Dataset URL: https://www.kaggle.com/datasets/victorsoeiro/netflix-tv-shows-and-movies
license(s): CC0-1.0
Downloading netflix-tv-shows-and-movies.zip to /content/data
  0% 0.00/2.25M [00:00<?, ?B/s]
100% 2.25M/2.25M [00:00<00:00, 795MB/s]
Archive:  netflix-tv-shows-and-movies.zip
  inflating: credits.csv
  inflating: titles.csv
credits.csv  netflix-tv-shows-and-movies.zip  titles.csv
```

ngrok Connection:
A free ngrok account must be created and an auth token generated in advance.

- NGROK_AUTHTOKEN

** The JMeter file with the pre-configured test plan is included as an attachment.*



4.2 ngrok

Configure `ngrok` authentication and open a public tunnel for local services on port `5000`.

[25] ✓ 5 s

```
from pyngrok import ngrok

ngrok_authtoken = userdata.get('NGROK_AUTHTOKEN')
ngrok.set_auth_token(ngrok_authtoken)
del ngrok_authtoken

public_url = ngrok.connect(5000)
print(f"Public URL: {public_url}")
```

6. Bibliography

MongoDB Documentation — https://www.mongodb.com/docs
MongoDB Atlas — https://www.mongodb.com/atlas
Kaggle Datasets — https://www.kaggle.com
Netflix Movies and TV Shows —
https://www.kaggle.com/datasets/victorsoeiro/netflix-tv-shows-and-movies
Student, W. (1908). The probable error of a mean. Biometrika.
Python Documentation — https://docs.python.org
Datablist — customers, organizations —
https://www.datablist.com/learn/csv/download-sample-csv-files
Apache JMeter Documentation — https://jmeter.apache.org/documentation.html

7. List of Attachments

- mongodb-performance-analysis-EN.pdf — project documentation (PDF)
- mongo-atlas-performance-test_netflix-db.ipynb — Jupyter notebook with analysis
- titles.csv — dataset
- credits.csv — dataset
- mongodb-performance-test.jmx — JMeter file with test plan configuration
- jmeter_netflix-db — folder containing .csv files with test results